



BURSA TEKNİK ÜNİVERSİTESİ

SAFeworker

İş Sağlığı ve Güvenliği İçin Mobil Risk Algılama Platformu

Ders	Node.js ile Web Programlama
Proje	İş Sağlığı ve Güvenliği İçin Mobil Risk Algılama
Ekip Üyeleri	Elif Nur Beycan 24360859210 Nermin Baycan 21360859019 Fatma Nur Yazıcı 22360859052 Kübra Kaya 23360859736
GitHub Deposu	https://github.com/elifnurbeycan/SafeWorker

İçindekiler

Özet.....	4
1. Proje Tanımı ve Giriş.....	5
1.1. Projenin Amacı.....	5
1.2. Problem Tanımı.....	5
1.3. Projenin Kapsamı ve Sınırlılıkları	5
2. Gereksinim Analizi.....	6
2.1. Paydaşlar ve Kullanıcı Rollerini	6
2.2. Kullanım Senaryoları.....	6
2.3. Föydeki Zorunlu Modüller ve Projedeki Karşılıkları.....	8
2.4. Fonksiyonel Gereksinimler.....	8
2.5. Fonksiyonel Olmayan Gereksinimler	9
2.6. Karşılaşılan Kısıtlar ve Varsayımlar.....	9
3. Sistem Mimarisi.....	10
3.1. Genel Mimari	10
3.2. Sensör Verisi ve Alarm Sequence Diyagramı.....	11
3.3. Backend Mimarisi	12
3.4. Mobil Uygulama Mimarisi	12
3.5. Dashboard Mimarisi	12
3.6. Veri Akışı.....	13
3.7. Gerçek Zamanlı Bildirim Akışı	14
4. Kullanılan Teknolojiler.....	15
5. Veritabanı Tasarımı	16
5.1. Veri Modeli Genel Bakışı.....	16
5.2. Koleksiyonlar ve Alanlar.....	17
5.3. Veri Saklama ve İlişki Mantığı.....	17
6. Backend Tasarımı ve API.....	19
6.1. API Endpointleri	19
6.2. Kimlik Doğrulama ve Yetkilendirme	19
6.3. Veri Doğrulama ve Hata Yönetimi.....	19
6.4. Swagger / OpenAPI Dokümantasyonu	19
6.5. Seed Verileri.....	20
7. Mobil Uygulama Tasarımı.....	21
7.1. Mobil Uygulama Akışı.....	21
7.2. Worker Login	22
7.3. Vardiya Başlatma ve Durdurma	23
7.4. Otomatik Sensör Veri Gönderimi	23
7.5. SOS / Acil Durum Bildirimi	23
7.6. QR Kod ile Tehlikeli Bölge Girişi.....	25

7.7. GPS Konum Desteđi	26
8. Dashboard Tasarımı.....	27
8.1. Dashboard Genel Görünüm	27
8.2. Son Durum	27
8.3. Çalışan Takibi	28
8.4. Alarm Listesi ve Filtreleme	28
8.5. Risk Grafiđi ve Tablo Gösterimleri	29
8.6. Sesli Kritik Alarm Sistemi.....	30
9. Risk Analizi ve Alarm Algoritması	31
9.1. Eşik Tabanlı Risk Analizi.....	31
9.2. Risk Puanı Hesaplama	31
9.3. Risk Seviyeleri	31
9.4. Alarm Türleri.....	31
9.5. Hareketsizlik Mantığının Gerekçesi	32
10. Gerçek Zamanlı İzleme	33
10.1. Socket.IO Kullanımı	33
10.2. Dashboard Canlı Güncelleme	33
11. Gerçekleştirilen Modüller ve Bonus Özellikler	34
12. Kurulum ve Çalıştırma	35
12.1. Gereksinimler	35
12.2. Backend Kurulumu.....	35
12.3. Dashboard Kurulumu	35
12.4. Mobil Uygulama Kurulumu	35
12.5. Docker ile Çalıştırma.....	35
13. Test Süreci ve Doğrulama	36
13.1. Test Ortamı.....	36
13.2. Test Kullanıcıları	36
13.3. Test Senaryoları.....	36
14. Ekip Görev Dağılımı	37
15. Sonuç ve Deđerlendirme	37
15.1. Projenin Genel Başarısı	37
15.2. Kazanımlar	37
15.3. Gelecek Geliştirmeler.....	37
Kaynakça.....	38

Özet

SafeWorker MVP, iş sağlığı ve güvenliği alanında saha çalışanlarının mobil cihazlarından alınan sensör verilerini kullanarak riskli durumları tespit etmeyi amaçlayan bir mobil risk algılama ve alarm yönetim platformudur. Proje; Flutter tabanlı mobil uygulama, Node.js/Express.js tabanlı backend, MongoDB veritabanı ve React tabanlı admin dashboard olmak üzere dört temel bileşenden oluşmaktadır. Worker kullanıcısı mobil uygulama üzerinden giriş yapar, vardiya başlatır ve vardiya aktif olduğu sürece mobil cihazdan alınan ivmeölçer, jiroskop, pil, ağ ve GPS verileri belirli aralıklarla backend sistemine gönderilir. Backend gelen verileri doğrular, MongoDB'ye kaydeder ve eşik tabanlı risk analizi uygulayarak risk puanı ve risk seviyesi üretir. Riskli durumlarda alarm oluşturulur ve alarmlar Socket.IO aracılığıyla admin dashboard üzerinde gerçek zamanlı olarak görüntülenir. Admin kullanıcısı dashboard üzerinden canlı çalışan durumlarını, risk grafiğini, alarm listesini, tehlikeli bölgeleri ve CSV raporlarını takip edebilir. Sistem ayrıca SOS bildirimi, QR kod ile tehlikeli bölge girişi, sesli kritik alarm, Swagger API dokümantasyonu ve Docker desteği gibi ek özellikler içermektedir. Bu çalışma, üretim ortamına hazır bir ticari ürün yerine temel işlevleri çalışan, geliştirilebilir ve test edilebilir bir minimum uygulanabilir ürün (MVP) olarak tasarlanmıştır.

1. Proje Tanımı ve Giriş

1.1. Projenin Amacı

SafeWorker projesinin amacı, saha çalışanlarının mobil cihazlarından alınan sensör verilerini kullanarak iş sağlığı ve güvenliği açısından riskli durumları erken tespit edebilen bir sistem geliştirmektir. Projede mobil telefon, ek bir IoT cihazına ihtiyaç duymadan veri toplayabilen bir sensör kaynağı olarak kullanılmıştır. Çalışan vardiya başlattığında uygulama otomatik olarak sensör verilerini toplar ve backend'e iletir. Backend bu verileri analiz ederek risk puanı üretir ve riskli durumlarda alarm oluşturur. Böylece iş güvenliği sorumlusu, dashboard üzerinden çalışanların durumunu canlı olarak izleyebilir.

1.2. Problem Tanımı

Saha çalışanlarının düşme, sert darbe, uzun süre hareketsiz kalma, tehlikeli bölgeye giriş, düşük pil veya bağlantı kaybı gibi durumları zamanında fark edilemeyebilir. Özellikle çalışanın yalnız görev yaptığı alanlarda bu durum müdahale süresini uzatabilir ve iş kazalarının etkisini artırabilir. Geleneksel takip yöntemleri çoğu zaman manuel gözlem veya periyodik kontrol üzerine kuruludur. Bu yöntemler anlık veri sağlamadığı için kritik olayların geç fark edilmesine neden olabilir. SafeWorker, mobil cihaz sensörlerinden yararlanarak bu problemi daha hızlı ve izlenebilir bir şekilde çözmeyi hedeflemektedir.

1.3. Projenin Kapsamı ve Sınırlılıkları

Proje kapsamında mobil sensör verisi toplama, vardiya bazlı veri gönderimi, backend risk analizi, alarm üretimi, dashboard izleme, QR tehlikeli bölge girişi, SOS bildirimi, gerçek zamanlı Socket.IO güncellemesi ve CSV alarm dışı aktarma özellikleri yer almaktadır. Sistem eğitim ve demo amacıyla geliştirilmiş bir MVP olduğu için üretim seviyesi güvenlik sertifikasyonu, büyük ölçekli saha dağıtımı, kamera/mikrofon analizi ve makine öğrenmesi tabanlı gelişmiş risk sınıflandırması bu sürümün kapsamı dışındadır.

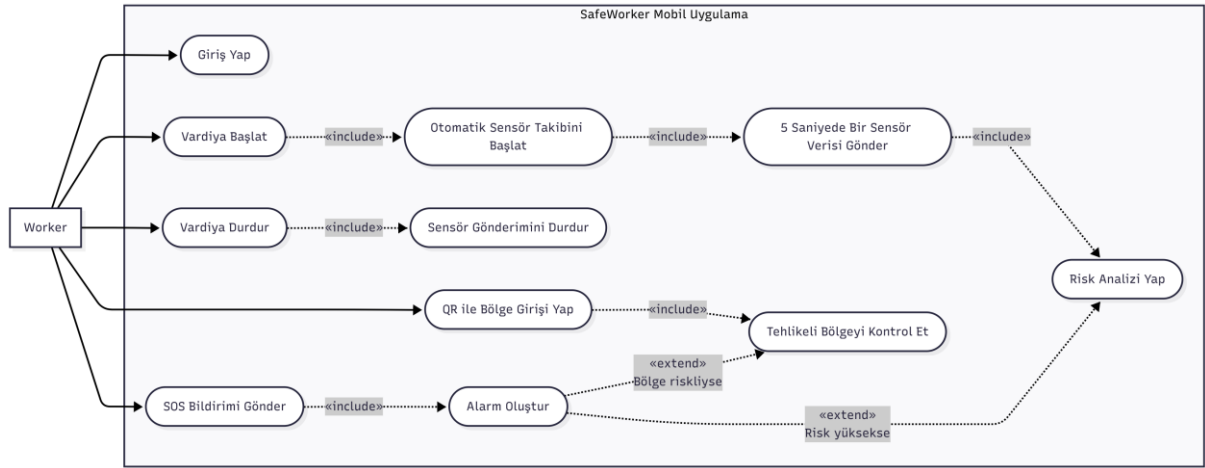
2. Gereksinim Analizi

2.1. Paydaşlar ve Kullanıcı Roller

Rol / Paydaş	Sistemdeki Görevi	Temel Yetkileri
Admin / İSG Sorumlusu	Dashboard üzerinden çalışanları, alarm kayıtlarını, risk durumlarını ve tehlikeli bölgeleri izler.	Canlı çalışan izleme, alarm listeleme, alarm çözme, CSV rapor alma, risk grafiği ve bölgeleri görüntüleme.
Worker / Saha Çalışanı	Mobil uygulama üzerinden sisteme giriş yapar, vardiya başlatır ve risk bildirimi oluşturabilir.	Giriş yapma, vardiya başlatma/durdurma, otomatik sensör veri akışı, QR okutma, SOS gönderme.

2.2. Kullanım Senaryoları

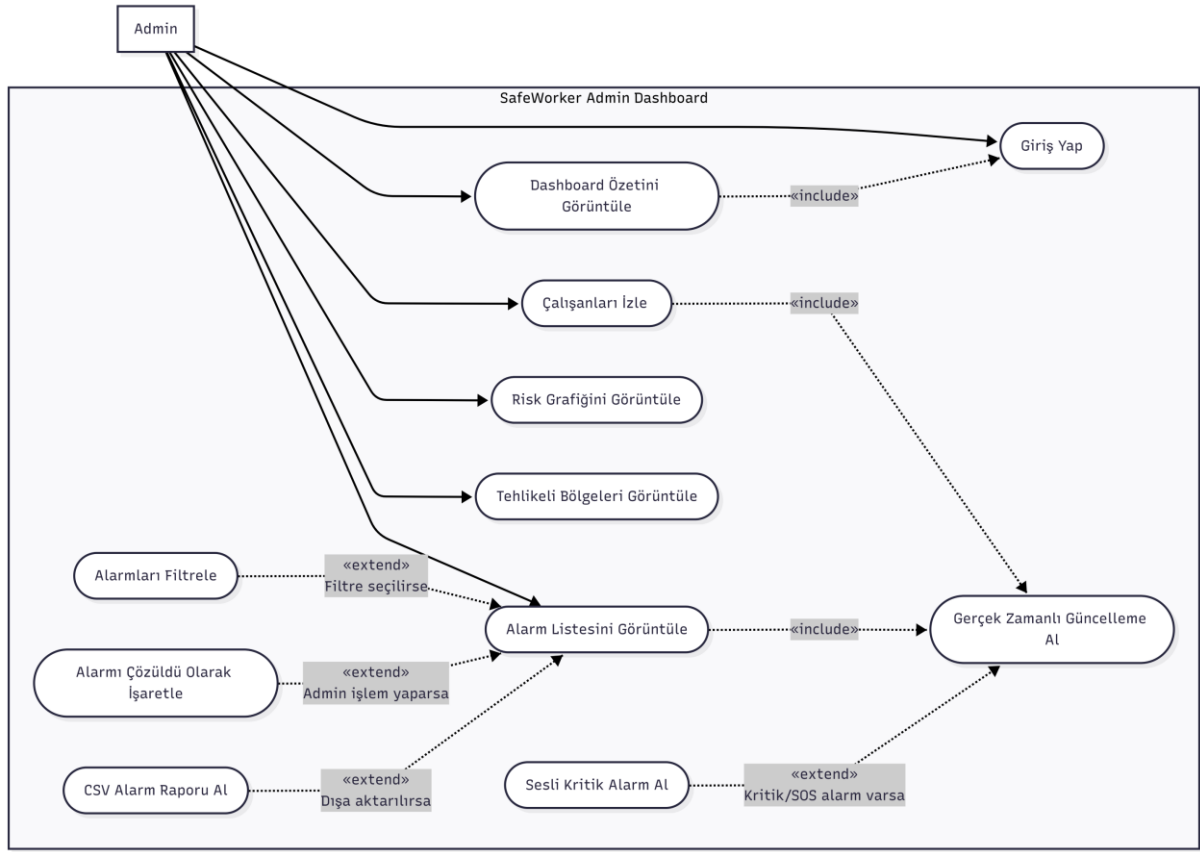
Use-case diyagramları, sistemdeki aktörlerin hangi işlemleri gerçekleştirdiğini ve bazı işlemlerin birbiriyle olan include/extend ilişkilerini göstermektedir. SafeWorker için Worker mobil uygulama işlemleri ve Admin dashboard işlemleri ayrı diyagramlarda verilmiştir. Böylece hem mobil taraftaki vardiya ve otomatik sensör mantığı hem de dashboard tarafındaki izleme ve alarm yönetimi daha okunabilir hale getirilmiştir.



Şekil 1. Worker Mobil Uygulama Use-Case Diyagramı

Şekil 1. Worker Mobil Uygulama Use-Case Diyagramı

Şekil 1’de Worker kullanıcısının mobil uygulama üzerinden gerçekleştirdiği temel işlemler gösterilmiştir. Worker doğrudan giriş yapma, vardiya başlatma, vardiya durdurma, QR ile bölge girişi yapma ve SOS bildirimi gönderme işlemlerini gerçekleştirir. Güncel sistemde çalışan sensör verisini manuel olarak göndermez; vardiya başlatıldığında sistem otomatik sensör takibini başlatır ve belirli aralıklarla sensör verisi backend’e gönderilir. Vardiya durdurulduğunda ise sensör gönderimi kesilir. Sensör verisi gönderimi risk analizi sürecini içerir; risk yüksekse alarm oluşturma durumu devreye girer.



Şekil 2. Admin use-Case Diyagramı

Şekil 2. Admin Dashboard Use-Case Diyagramı

Şekil 2’de Admin kullanıcısının dashboard üzerinden gerçekleştirdiği işlemler gösterilmiştir. Admin sisteme giriş yaptıktan sonra dashboard özetini, çalışanları, risk grafiğini, tehlikeli bölgeleri ve alarm listesini görüntüleyebilir. Alarm listesi üzerinden filtreleme, alarmı çözüldü olarak işaretleme ve CSV alarm raporu alma işlemleri yapılabilir. Çalışan ve alarm verileri gerçek zamanlı güncelleme süreciyle desteklenir. Kritik veya SOS alarmı oluştuğunda dashboard üzerinde sesli kritik alarm özelliği devreye girebilir.

2.3. Föydeki Zorunlu Modüller ve Projedeki Karşılıkları

Modül	SafeWorker Karşılığı	Kanıt / Rapor Bölümü
Mobil veri toplama	Flutter uygulama; ivmeölçer, jiroskop, pil, ağ ve GPS verisi gönderimi.	Mobil uygulama tasarımı, vardiya akışı ve sensör gönderim açıklamaları.
Node.js backend	Express.js API, controller-route-service yapısı, doğrulama ve hata yönetimi.	Backend tasarımı ve API bölümü.
Kullanıcı doğrulama ve yetkilendirme	JWT, bcrypt, admin/worker rolleri ve role middleware mantığı.	Kimlik doğrulama ve yetkilendirme bölümü.
Veritabanı	MongoDB + Mongoose; User, Device, Shift, SensorData, Alarm, DangerZone, ZoneEntry.	ER diyagramı ve veri modeli bölümü.
Gerçek zamanlı izleme paneli	React dashboard ve Socket.IO eventleri.	Sistem mimarisi, DFD, sequence ve dashboard bölümleri.
Analiz / anomali tespiti	Eşik tabanlı risk puanı ve risk seviyesi hesaplama.	Risk analizi akış diyagramı ve algoritma bölümü.
Alarm sistemi	Hard impact, fall risk, inactivity, low battery, connection lost, SOS ve danger zone entry alarmları.	Alarm türleri ve dashboard alarm listesi açıklamaları.
Dokümantasyon	README, Swagger/OpenAPI, teknik rapor ve test senaryoları.	Swagger, GitHub ve teslim dosyaları bölümleri.

2.4. Fonksiyonel Gereksinimler

Kod	Gereksinim	Öncelik	Durum
FR-01	Kullanıcı e-posta ve şifre ile giriş yapabilmelidir.	Yüksek	Tamamlandı
FR-02	Sistem admin ve worker rollerini ayırmalıdır.	Yüksek	Tamamlandı
FR-03	Worker mobil uygulamadan vardiya başlatıp durdurabilmelidir.	Yüksek	Tamamlandı
FR-04	Mobil uygulama vardiya aktifken sensör verilerini otomatik aralıklarla backend'e göndermelidir.	Yüksek	Tamamlandı
FR-05	Backend gelen sensör verilerinden risk puanı hesaplamalıdır.	Yüksek	Tamamlandı
FR-06	Riskli durumda alarm oluşturulmalı ve MongoDB'ye kaydedilmelidir.	Yüksek	Tamamlandı
FR-07	Admin dashboard canlı çalışan, risk ve alarm verilerini göstermelidir.	Yüksek	Tamamlandı
FR-08	SOS bildirimi dashboard'da kritik alarm olarak görünmelidir.	Yüksek	Tamamlandı
FR-09	QR kod ile tehlikeli bölge girişi kaydedilmelidir.	Orta	Tamamlandı
FR-10	Alarm kayıtları CSV olarak dışa aktarılabilir.	Orta	Tamamlandı

2.5. Fonksiyonel Olmayan Gereksinimler

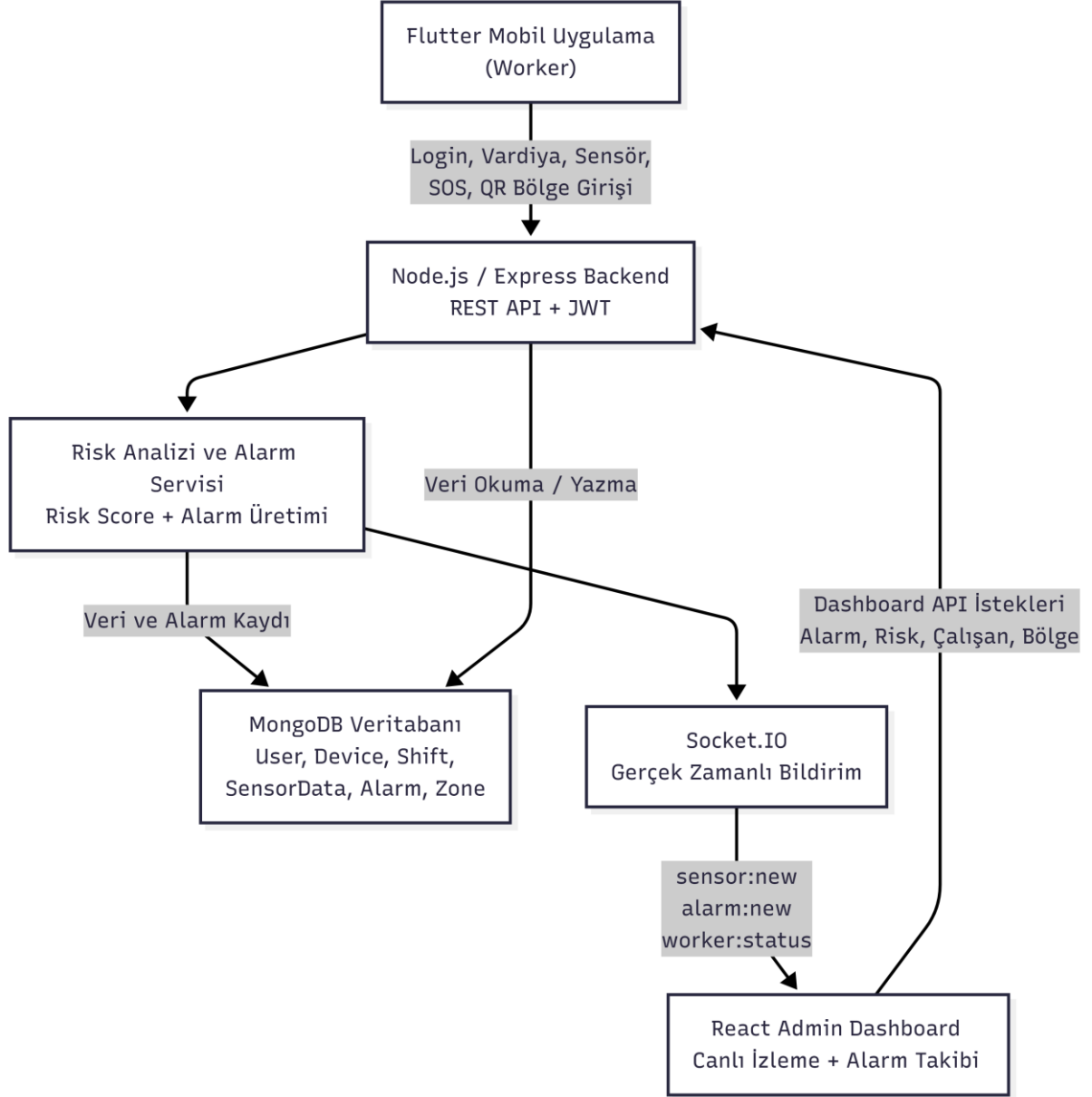
Kod	Gereksinim	Açıklama
NFR-01	Güvenlik	Şifreler hashlenmeli, işlemler JWT ile korunmalı ve rol bazlı erişim uygulanmalıdır.
NFR-02	Gerçek zamanlılık	Yeni sensör verisi, worker durumu ve alarmlar dashboard'a sayfa yenilemeden yansımalıdır.
NFR-03	Kullanılabilirlik	Mobil uygulama worker için sade, dashboard admin için anlaşılır ve takip edilebilir olmalıdır.
NFR-04	Bakım yapılabilirlik	Backend, dashboard ve mobile klasörleri modüler biçimde ayrılmalıdır.
NFR-05	Taşınabilirlik	Sistem yerel kurulumla ve isteğe bağlı Docker Compose ile çalıştırılabilir olmalıdır.

2.6. Karşılaşılan Kısıtlar ve Varsayımlar

- Backend ve mobil cihaz aynı ağ üzerinde çalıştırıldığında mobil uygulamanın bilgisayardaki backend IP adresine erişebilmesi gerekir.
- MongoDB bağlantısının aktif olması, sensör ve alarm kayıtlarının saklanabilmesi için zorunludur.
- Mobil uygulamanın sensör, konum ve ağ bilgilerine erişebilmesi için gerekli Android izinlerinin verilmesi gerekir.
- Dashboard üzerindeki sesli alarmin çalışabilmesi için tarayıcının ses oynatma izni vermesi gerekebilir.
- Proje MVP olduğu için gerçek saha koşullarındaki uzun süreli performans, güvenlik sertifikasyonu ve büyük ölçekli dağıtım bu raporda kapsam dışıdır.

3. Sistem Mimarisi

3.1. Genel Mimari

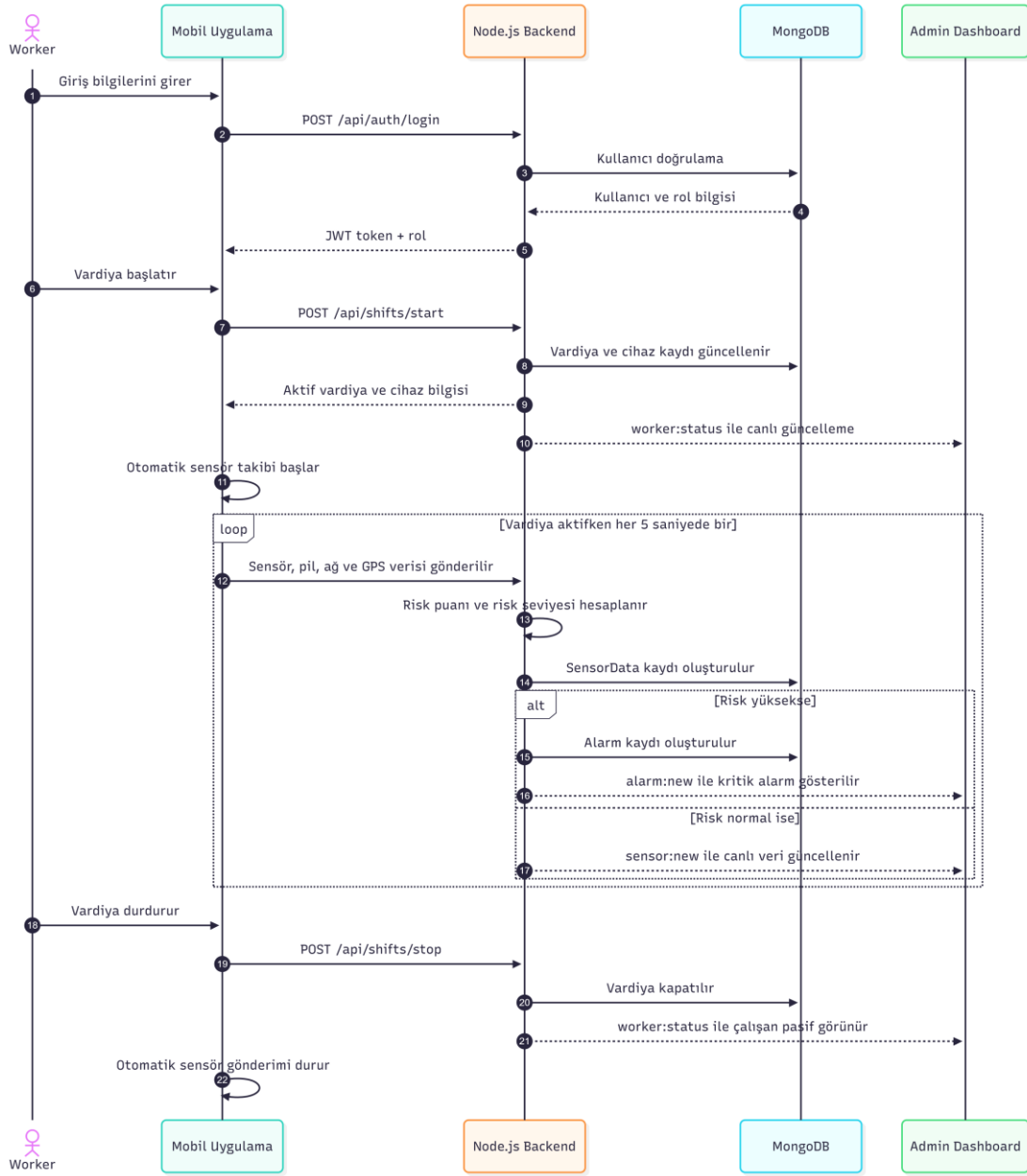


Şekil 3. SafeWorker Sistem Mimarisi

Şekil 3. SafeWorker Genel Sistem Mimarisi

Şekil 3'te SafeWorker sisteminin genel mimarisi gösterilmiştir. Sistem; Flutter mobil uygulama, Node.js/Express.js backend, MongoDB veritabanı, Socket.IO gerçek zamanlı bildirim yapısı ve React admin dashboard bileşenlerinden oluşmaktadır. Worker mobil uygulama üzerinden giriş, vardiya, sensör, SOS ve QR bölge girişi verilerini backend'e gönderir. Backend gelen verileri doğrular, risk analizi ve alarm üretimi yapar, verileri MongoDB'de saklar ve Socket.IO ile dashboard'u canlı olarak günceller. Admin dashboard ise canlı çalışan takibi, alarm yönetimi ve risk izleme arayüzü olarak görev yapar.

3.2. Sensör Verisi ve Alarm Sequence Diyagramı



Şekil 4. Sequence Diyagramı

Şekil 4. SafeWorker Sequence Diyagramı

Şekil 4'te SafeWorker sisteminde Worker kullanıcısının giriş yapması, vardiya başlatması, mobil uygulamanın otomatik sensör verisi göndermesi, backend'in risk analizi yapması, alarm oluşması ve dashboard'un gerçek zamanlı güncellenmesi sıralı olarak gösterilmiştir. Worker sensör verisini manuel olarak göndermez; vardiya aktif olduğu sürece mobil uygulama belirli aralıklarla sensör, pil, ağ ve GPS verisini backend'e iletir. Risk seviyesi yüksekse alarm kaydı oluşturulur ve admin dashboard üzerinde alarm:new olayıyla canlı olarak görüntülenir. Risk normal ise sensor:new olayıyla canlı veri güncellemesi yapılır.

3.3. Backend Mimarisi

Backend katmanı Node.js ve Express.js üzerine kurulmuştur. API endpointleri route dosyalarında tanımlanır, gelen HTTP istekleri controller katmanında karşılanır ve iş mantığı service katmanında yürütülür. MongoDB koleksiyonları model katmanında Mongoose şemaları ile temsil edilir. Kimlik doğrulama, rol kontrolü ve merkezi hata yönetimi ise middleware yapısıyla gerçekleştirilir. Bu ayırım, kodun daha modüler, anlaşılır ve geliştirilebilir olmasını sağlar.

Katman	Görev	Örnek
Routes	API endpointlerini tanımlar.	/api/auth, /api/sensor-data, /api/alarms, /api/zones
Controllers	HTTP isteğini karşılar ve servisleri çağırır.	login, sensorDataCreate, alarmResolve
Services	İş mantığını yürütür.	risk analizi, alarm oluşturma, worker status
Models	MongoDB koleksiyon şemalarını tanımlar.	User, Device, Shift, SensorData, Alarm
Middlewares	Kimlik doğrulama, rol kontrolü ve hata yönetimi yapar.	auth, role, errorHandler

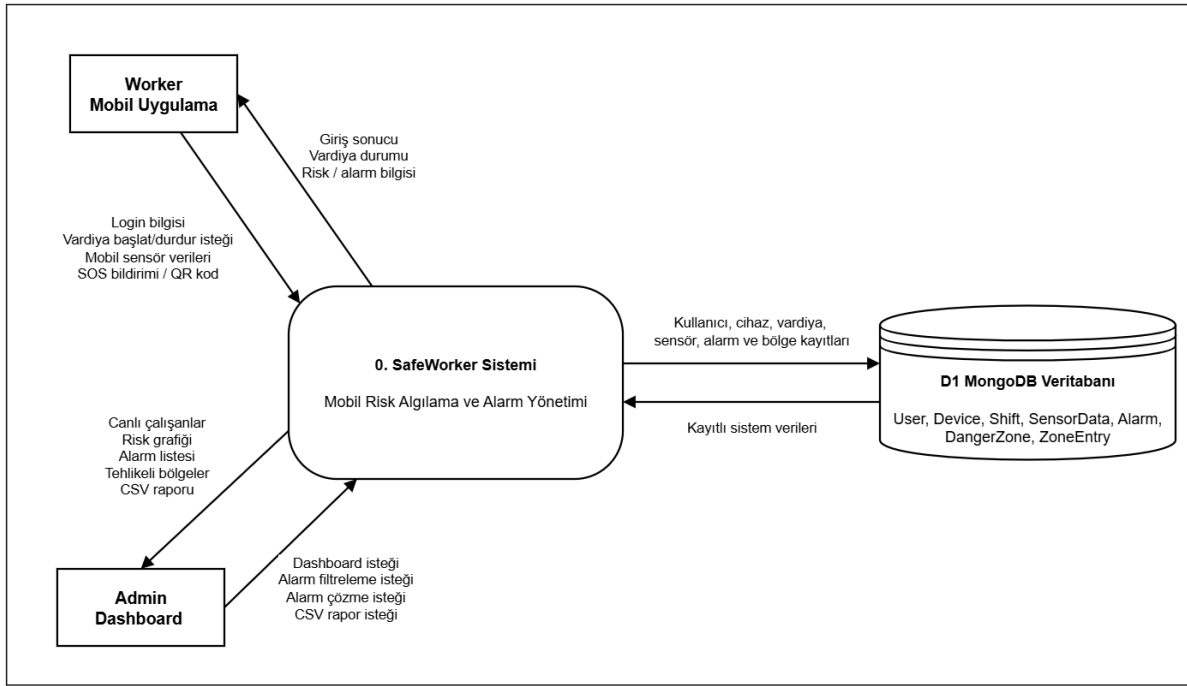
3.4. Mobil Uygulama Mimarisi

Mobil uygulama Flutter ile geliştirilmiştir. Uygulamada worker login, ana ekran, vardiya başlatma/durdurma, sensör takip, SOS bildirimi ve QR bölge girişi akışları yer almaktadır. Token ve kullanıcı bilgileri local storage/shared preferences üzerinde tutulur. Vardiya başlatıldığında uygulama otomatik sensör takibini başlatır ve belirlenen aralıklarla backend'e sensör verisi gönderir. Bu yapı, çalışanın manuel olarak veri göndermesine gerek kalmadan vardiya süresince düzenli veri akışı sağlar.

3.5. Dashboard Mimarisi

Dashboard katmanı React ve Vite kullanılarak geliştirilmiştir. Admin kullanıcısı dashboard üzerinden sistem özet kartlarını, canlı çalışanları, alarm listesini, risk grafiğini ve tehlikeli bölge bilgilerini görüntüler. Dashboard, REST API üzerinden geçmiş ve özet verileri alırken Socket.IO client yapısıyla sensor:new, alarm:new ve worker:status olaylarını dinler. Bu sayede dashboard sayfa yenilemeye gerek kalmadan canlı olarak güncellenir.

3.6. Veri Akışı



Şekil 5. SafeWorker DFD Seviye 0 Genel Veri Akış Diyagramı

Şekil 5. SafeWorker DFD Seviye 0 Genel Veri Akış Diyagramı

Şekil 5'te SafeWorker sisteminin DFD Seviye 0 genel veri akış diyagramı gösterilmiştir. Worker mobil uygulama üzerinden login bilgisi, vardiya başlat/durdur isteği, mobil sensör verileri, SOS bildirimi ve QR kod bilgisini sisteme gönderir. Sistem kullanıcıya giriş sonucu, vardiya durumu ve risk/alarm bilgisini döndürür. Admin dashboard üzerinden canlı çalışanlar, risk grafiği, alarm listesi, tehlikeli bölgeler ve CSV raporu gibi çıktıları alırken; dashboard isteği, alarm filtreleme, alarm çözme ve CSV rapor isteği gönderir. Tüm kullanıcı, cihaz, vardiya, sensör, alarm ve bölge verileri MongoDB veritabanında saklanır.

```

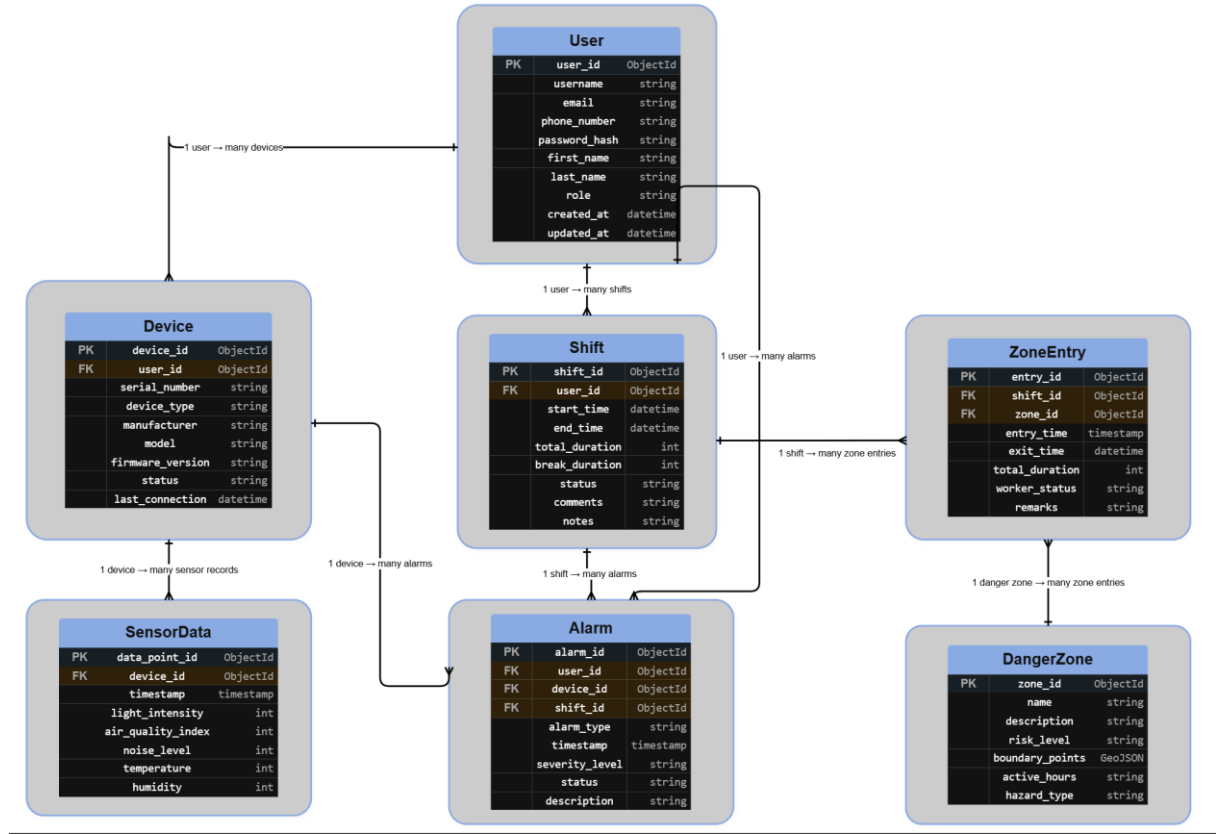
graph TD
    Worker[Worker Mobil Uygulama] -- "QR kod" --> D1[D1 User, Device, Shift]
    Worker -- "vardiya başlat/durdur" --> D1
    Worker -- "login bilgisi" --> 1[1.0 Kimlik Doğrulama]
    1 -- "token / rol" --> Worker
    1 -- "aktif cihaz / vardiya bilgisi" --> Worker
    1 -- "worker.status" --> 2[2.0 Vardiya ve Cihaz Yönetimi]
    2 -- "kayıtlı kullanıcı" --> D1
    2 -- "kullanıcı doğrulama" --> D1
    2 -- "cihaz + vardiya kaydı" --> D1
    2 -- "aktif cihaz vardiya bilgisi" --> D1
    D1 -- "dashboard için kayıtlı veriler" --> 5[5.0 Alarm Yönetimi]
    3[3.0 Mobil Veri Toplama (5 sn otomatik)] -- "sensör, pil, ağ, GPS" --> Worker
    3 -- "SOS bildirimi" --> Worker
    3 -- "sensör kaydı" --> 4[4.0 Risk Analizi]
    4 -- "risk puanı / seviye" --> 5
    4 -- "risk yüksekse" --> 5
    5 -- "alarm kaydı" --> D2[D2 SensorData, Alarm]
    5 -- "alarm durumu" --> D2
    D2 -- "sensör ve alarm verileri" --> 6[6.0 Dashboard ve Raporlama]
    D2 -- "bölge bilgisi risk durumu" --> D3[D3 DangerZone, ZoneEntry]
    D3 -- "riskli bölge alarmı" --> 5
    D3 -- "bölge verileri" --> 6
    6 -- "bölge bilgisi risk durumu" --> D3
    6 -- "canlı güncelleme" --> 7[7.0 Gerçek Zamanlı Bildirim / Socket.IO]
    7 -- "anlık bildirim" --> Admin[Admin Dashboard]
    7 -- "dashboard isteği / filtre / çözme / CSV" --> Admin
    Admin -- "çalışanlar / alarm / risk / CSV" --> 6
    
```

4. Kullanılan Teknolojiler

Bileşen	Teknoloji	Kullanım Amacı
Backend	Node.js, Express.js	REST API ve iş mantığı
Veritabanı	MongoDB, Mongoose	Kullanıcı, cihaz, vardiya, sensör ve alarm kayıtları
Güvenlik	JWT, bcrypt	Kimlik doğrulama, şifre güvenliği ve rol bazlı erişim
Gerçek zamanlı iletişim	Socket.IO	Dashboard canlı güncelleme ve alarm bildirimi
Dashboard	React, Vite, Recharts	Admin paneli, grafik ve tablo gösterimi
Mobil	Flutter, sensors_plus, battery_plus, connectivity_plus, geolocator	Sensör, pil, ağ ve GPS verisi toplama
Dokümantasyon	Swagger / OpenAPI	API endpointlerinin açıklanması ve test edilmesi
Dağıtım desteği	Docker / Docker Compose	Backend, dashboard ve MongoDB servislerinin birlikte çalıştırılması

5. Veritabanı Tasarımı

5.1. Veri Modeli Genel Bakışı



Şekil 7. ER _ Veri Tabanı Diyagramı

Şekil 7. SafeWorker ER / Veri Tabanı Diyagramı

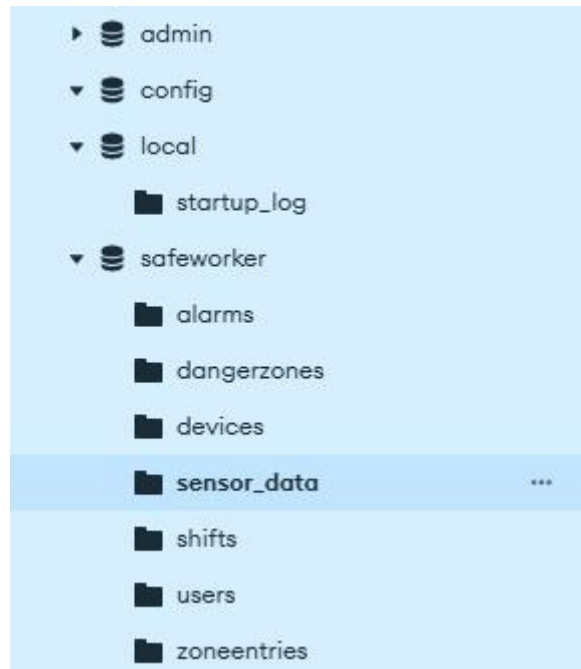
Şekil 7’de SafeWorker sisteminde kullanılan temel MongoDB koleksiyonları ve aralarındaki ilişkiler gösterilmiştir. User koleksiyonu kullanıcı bilgilerini, Device koleksiyonu çalışana bağlı cihaz bilgilerini, Shift koleksiyonu vardiya durumunu, SensorData koleksiyonu mobil cihazdan gelen ölçümleri, Alarm koleksiyonu riskli durum kayıtlarını, DangerZone koleksiyonu tehlikeli bölge tanımlarını ve ZoneEntry koleksiyonu QR kod ile yapılan bölge girişlerini saklar. Diyagramda bir worker’ın birden fazla cihaz, vardiya, sensör kaydı ve alarm ile ilişkili olabileceği; bir tehlikeli bölgenin ise birden fazla bölge giriş kaydıyla ilişkilendirilebileceği gösterilmiştir.

5.2. Koleksiyonlar ve Alanlar

Model	Temel Alanlar	Açıklama
User	name, email, password/passwordHash, role	Admin ve worker kullanıcı bilgilerini tutar.
Device	deviceCode/deviceId, deviceName, workerId, batteryLevel, networkStatus, lastSeen	Worker'a bağlı mobil cihaz kaydını ve güncel cihaz durumunu tutar.
Shift	workerId, deviceId, startTime, endTime, status	Worker'ın aktif çalışma/vardiya durumunu saklar.
SensorData	workerId, deviceId, shiftId, timestamp, accelerometer, gyroscope, batteryLevel, networkStatus, location, riskScore, riskLevel	Mobil cihazdan gelen ölçüm ve risk verilerini saklar.
Alarm	workerId, deviceId, shiftId, type, message, riskScore, status, resolvedAt	Riskli durumlarda oluşturulan alarm kayıtlarını saklar.
DangerZone	zoneName/name, qrCode, riskLevel, description, isActive	QR ile tanımlanan tehlikeli bölgeleri saklar.
ZoneEntry	workerId/shiftId, zoneId, entryTime/createdAt	Çalışanın QR kod ile bölge giriş kayıtlarını saklar.

5.3. Veri Saklama ve İlişki Mantığı

Sistemde worker, cihaz, vardiya ve sensör verileri birbirine referans alanlarıyla bağlanır. Worker giriş yaptıktan sonra aktif cihaz ve vardiya bilgisi backend tarafından belirlenir. Vardiya aktifken gelen sensör verileri ilgili worker, device ve shift bağlamında SensorData koleksiyonuna kaydedilir. Alarm kayıtları da aynı bağlamla tutulur; böylece bir alarmın hangi çalışana, hangi cihaza ve hangi vardiyaya ait olduğu izlenebilir. QR kod ile tehlikeli bölgeye giriş yapıldığında DangerZone ve ZoneEntry koleksiyonları kullanılarak bölge giriş kaydı oluşturulur.



Şekil 8. MongoDB Koleksiyonları

Şekil 8. MongoDB Koleksiyonları

Şekil 8’de SafeWorker sisteminde kullanılan MongoDB koleksiyonlarının görüntüsü yer almalıdır. Bu görsel, User, Device, Shift, SensorData, Alarm, DangerZone ve ZoneEntry gibi koleksiyonların veritabanında oluşturulduğunu ve mobil uygulamadan gelen verilerin zaman damgası ile birlikte kaydedilebildiğini kanıtlamak amacıyla kullanılacaktır.

6. Backend Tasarımı ve API

6.1. API Endpointleri

Modül	Endpoint	Amaç
Auth	POST /api/auth/login	Kullanıcı girişi ve JWT token üretimi.
Device	GET /api/devices/my	Worker kullanıcısına bağlı cihaz bilgisini alma.
Shift	POST /api/shifts/start	Worker vardiyasını başlatma.
Shift	PATCH /api/shifts/stop	Aktif vardiyayı durdurma.
Sensor Data	POST /api/sensor-data	Mobil sensör verisini backend'e gönderme.
Alarms	GET /api/alarms	Alarm geçmişini listeleme.
Alarms	GET /api/alarms/active	Aktif alarmları listeleme.
Alarms	PATCH /api/alarms/:id/resolve	Alarmı çözüldü olarak işaretleme.
Alarms	GET /api/alarms/export.csv	Alarm kayıtlarını CSV olarak dışa aktarma.
Emergency	POST /api/emergency	SOS / acil durum alarmı oluşturma.
Zones	POST /api/zones/scan	QR kod ile tehlikeli bölge girişini işleme.
Dashboard	GET /api/dashboard/summary	Dashboard özet metriklerini alma.
Dashboard	GET /api/dashboard/live-workers	Canlı çalışan durumlarını alma.

6.2. Kimlik Doğrulama ve Yetkilendirme

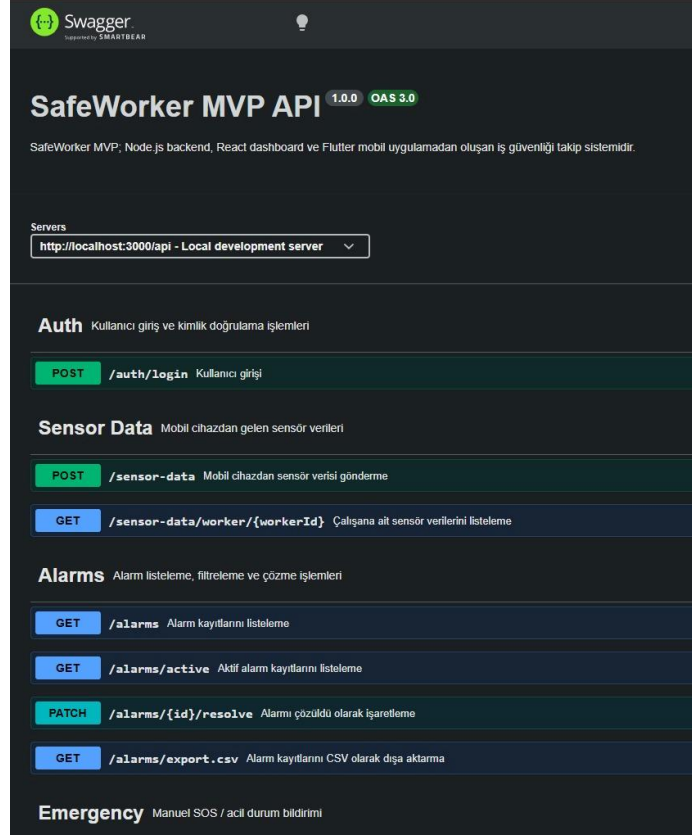
Kullanıcı sisteme e-posta ve şifre ile giriş yapar. Backend, kullanıcı bilgilerini doğruladıktan sonra JWT token üretir. Bu token sonraki API isteklerinde Authorization header içinde kullanılır. Sistem admin ve worker rollerini ayırır. Worker yalnızca kendi mobil işlemlerini yapabilirken, Admin dashboard ve alarm yönetimi gibi yönetimsel işlemlere erişebilir. Bu rol ayrımı, sistem güvenliği ve yetki sınırlarının korunması açısından önemlidir.

6.3. Veri Doğrulama ve Hata Yönetimi

Backend tarafında gelen verilerin eksik veya hatalı olması durumunda uygun hata mesajları döndürülür. Sensör verileri, vardiya bilgileri, cihaz bilgileri ve alarm işlemleri kontrol edilerek MongoDB'ye kaydedilir. Merkezi hata yönetimi sayesinde beklenmeyen durumlarda API tutarlı cevaplar üretir. Bu yapı, mobil uygulama ve dashboard tarafında hataların daha anlaşılır biçimde gösterilmesini sağlar.

6.4. Swagger / OpenAPI Dokümantasyonu

Projede Swagger/OpenAPI dokümantasyonu API endpointlerinin açıklanması ve test edilmesi için kullanılmıştır. Swagger arayüzü sayesinde login, sensor-data, emergency, zones/scan, alarms ve dashboard endpointleri tek bir arayüz üzerinden incelenebilir. Bu özellik föyde zorunlu olmasa da API'nin daha anlaşılır ve test edilebilir hale gelmesini sağlayan önemli bir bonus geliştirmedir.



Şekil 9. Swagger API Dokümantasyonu Ekranı

Şekil 9. Swagger Dokümantasyonu Ekranı

Şekil 9’da SafeWorker backend API endpointlerinin Swagger arayüzü üzerinde görüntülenmesi gösterilmelidir. Bu ekran; auth, sensor-data, emergency, zones, alarms ve dashboard endpointlerinin dokümante edildiğini, API isteklerinin tek bir arayüz üzerinden incelenebildiğini ve projenin teknik dokümantasyon yönünün güçlendirildiğini göstermektedir.

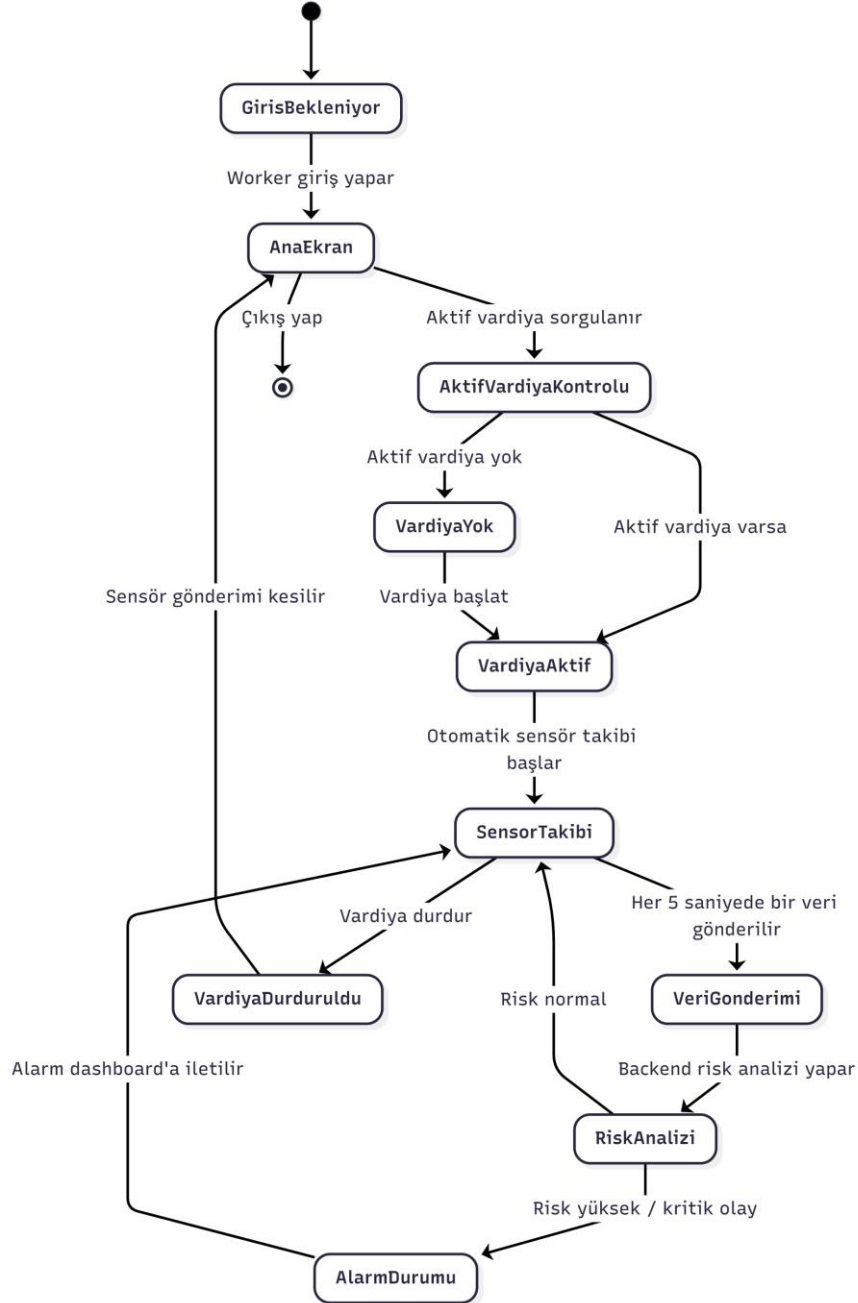
6.5. Seed Verileri

Sistemi test edebilmek için seed verileri kullanılmıştır. Seed script ile örnek admin ve worker kullanıcıları, örnek cihazlar, tehlikeli bölgeler ve başlangıç verileri oluşturulabilir. Bu sayede proje ilk kurulumdan sonra manuel veri girmeden test edilebilir hale gelir. Seed verileri özellikle dashboard, QR bölge girişi ve alarm testlerinde hızlı başlangıç sağlar.

7. Mobil Uygulama Tasarımı

7.1. Mobil Uygulama Akışı

Mobil uygulama Worker kullanıcısının sahada kullandığı ana arayüzdür. Uygulama üzerinden kullanıcı girişi, vardiya başlatma/durdurma, otomatik sensör takibi, SOS bildirimi ve QR kod ile tehlikeli bölge girişi yapılabilir. Mobil uygulamanın en önemli özelliği, sensör verisinin manuel değil, vardiya durumuna bağlı otomatik olarak gönderilmesidir.



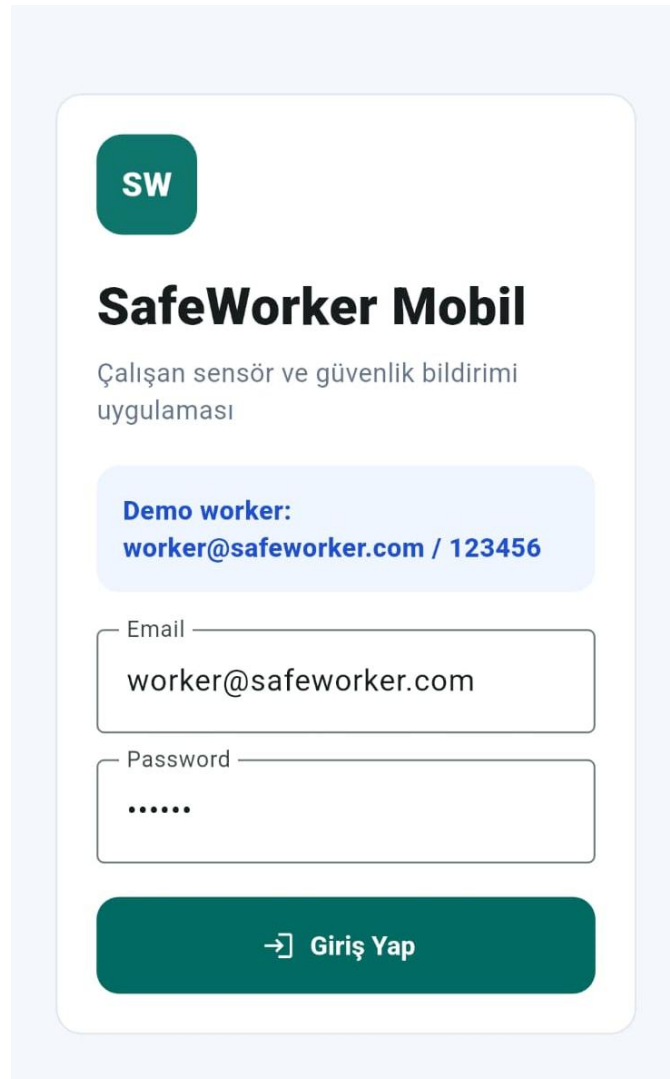
Şekil 10. Vardiya Durum Diyagramı

Şekil 10. SafeWorker Vardiya Durum Diyagramı

Şekil 10'da mobil uygulamadaki vardiya durum akışı gösterilmiştir. Worker sisteme giriş yaptıktan sonra aktif vardiya durumu kontrol edilir. Aktif vardiya yoksa kullanıcı vardiya başlatabilir; aktif vardiya varsa sistem doğrudan vardiya aktif durumuna geçer. Vardiya başlatıldığında otomatik sensör takibi başlar ve her 5 saniyede bir backend'e veri gönderilir. Backend gelen verileri risk analizinden geçirir. Risk normal ise sensör takibi devam eder, risk yüksek veya kritik olay varsa alarm dashboard'a iletilir. Worker vardiyayı durdurduğunda sensör gönderimi otomatik olarak kesilir.

7.2. Worker Login

Worker e-posta ve şifre ile mobil uygulamaya giriş yapar. Giriş başarılı olduğunda backend JWT token ve rol bilgisini döndürür. Bu bilgiler local storage/shared preferences üzerinde saklanır ve sonraki API isteklerinde kullanılır. Böylece worker'ın kimliği ve yetkisi backend tarafından doğrulanmış olur.

The image shows a mobile application login screen for 'SafeWorker Mobil'. At the top left is a green square icon with 'SW' in white. Below it, the title 'SafeWorker Mobil' is displayed in a large, bold, black font. Under the title, a subtitle reads 'Çalışan sensör ve güvenlik bildirimi uygulaması'. A light blue box contains the text 'Demo worker: worker@safeworker.com / 123456'. Below this, there are two input fields: 'Email' with the value 'worker@safeworker.com' and 'Password' with masked characters '.....'. At the bottom is a large green button with a white right-pointing arrow and the text 'Giriş Yap'.

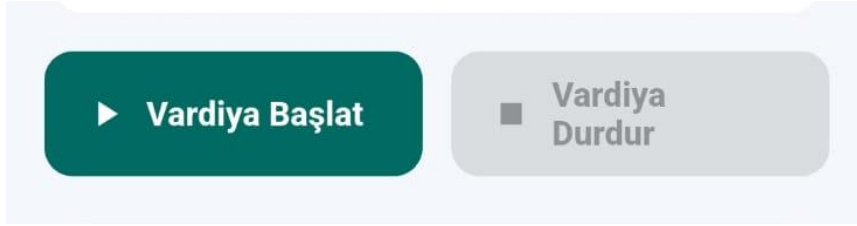
Şekil 11. Worker Mobil Giriş Ekranı

Şekil 11. Worker Mobil Giriş Ekranı

Şekil 11’de worker kullanıcısının mobil uygulama üzerinden e-posta ve şifre bilgileri ile sisteme giriş yaptığı ekran gösterilmelidir. Bu ekran, kullanıcı doğrulama sürecinin mobil taraftaki başlangıç noktasıdır. Giriş başarılı olduğunda backend JWT token ve rol bilgisi üretir; mobil uygulama bu bilgileri sonraki isteklerde kullanmak üzere saklar.

7.3. Vardiya Başlatma ve Durdurma

Worker vardiya başlattığında backend üzerinde aktif shift oluşturulur ve cihaz bilgisiyle ilişkilendirilir. Vardiya aktif hale geldiğinde mobil uygulama otomatik sensör takibini başlatır. Worker vardiyayı durdurduğunda backend shift durumunu kapatır ve mobil uygulamadaki sensör gönderim döngüsü durdurulur. Bu yapı, gerçek çalışma senaryosuna daha uygun bir veri akışı sağlar.



Şekil 12. Worker Vardiya Başlatma Ekranı

Şekil 12. Worker Vardiya Başlatma Ekranı

Şekil 12’de worker kullanıcısının mobil uygulama üzerinden vardiya başlatma işlemi gösterilmelidir. Vardiya başlatıldığında backend tarafında aktif shift kaydı oluşturulur, cihaz bilgisi worker ile ilişkilendirilir ve mobil uygulama otomatik sensör takibini başlatacak duruma geçer.

7.4. Otomatik Sensör Veri Gönderimi

Uygulama ivmeölçer, jiroskop, pil seviyesi, ağ durumu ve GPS konumu gibi verileri toplar. Vardiya aktif olduğu sürece bu veriler belirli aralıklarla, örneğin 5 saniyede bir, backend’e gönderilir. Bu veri gönderimi manuel butona bağlı değildir; worker vardiyayı başlattığında otomatik olarak başlar ve vardiya durdurulduğunda kesilir.

7.5. SOS / Acil Durum Bildirimi

Worker acil durumda SOS butonuna basarak backend’e acil durum bildirimi gönderebilir. Bu durumda sistem EMERGENCY_BUTTON tipinde alarm oluşturur. SOS bildirimi yüksek öncelikli kabul edilir ve dashboard üzerinde kritik alarm olarak görünür. Sesli kritik alarm özelliği aktifse admin kullanıcısı sesli uyarı da alabilir.



Manuel Acil Durum Bildirimi

Butona basıldığında backend üzerinde
EMERGENCY_BUTTON alarmı oluşturulur.

SOS Acil Durum Gönder

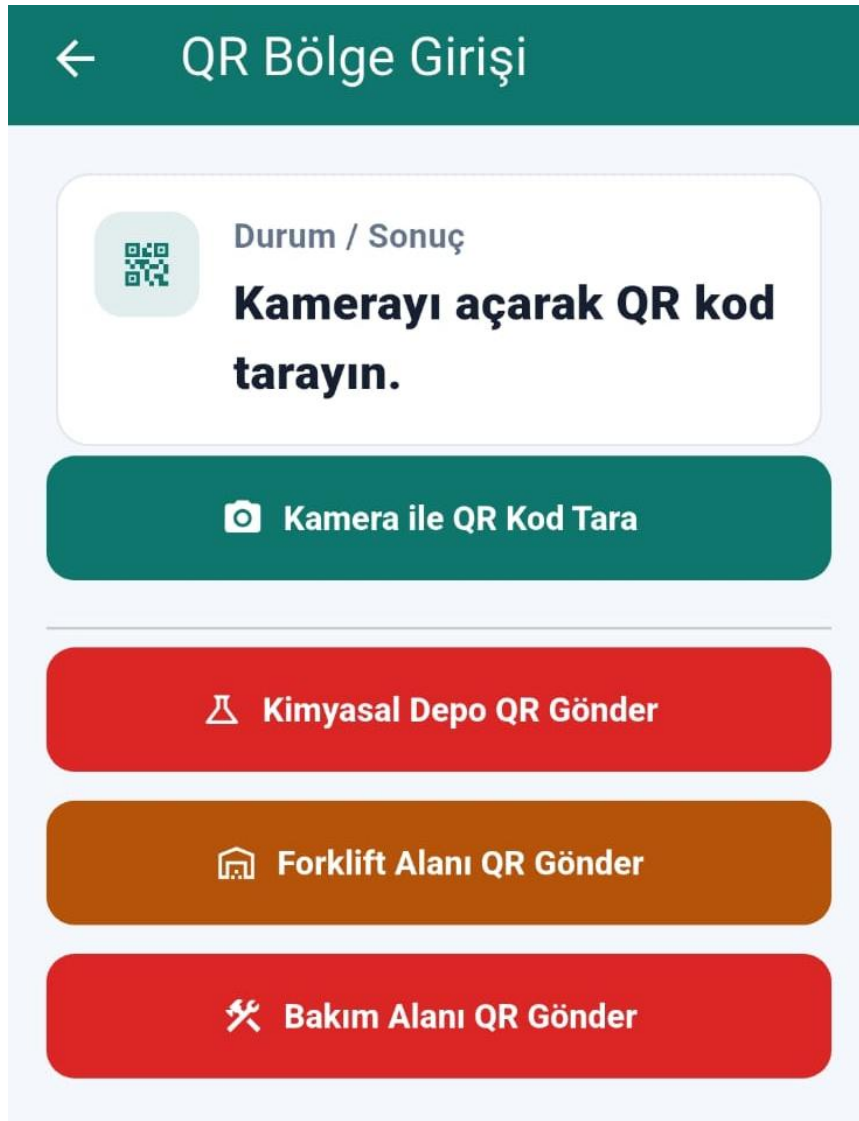
Şekil 13. SOS Bildirimi Ekranı

Şekil 13. SOS Bildirimi Ekranı

Şekil 13'te worker kullanıcısının acil durumda SOS bildirimi gönderebildiği mobil ekran gösterilmelidir. SOS işlemi, backend tarafında EMERGENCY_BUTTON tipinde yüksek öncelikli alarm oluşturur ve bu alarm dashboard üzerinde kritik durum olarak görüntülenir.

7.6. QR Kod ile Tehlikeli Bölge Girişi

Worker QR kod okutarak tehlikeli bölge girişini sisteme iletebilir. Backend QR kodu DangerZone kaydıyla eşleştirir ve bölge risk seviyesini kontrol eder. Bölge yüksek riskliyse DANGER_ZONE_ENTRY alarmı oluşturulur ve bölge giriş kaydı ZoneEntry koleksiyonunda saklanır.



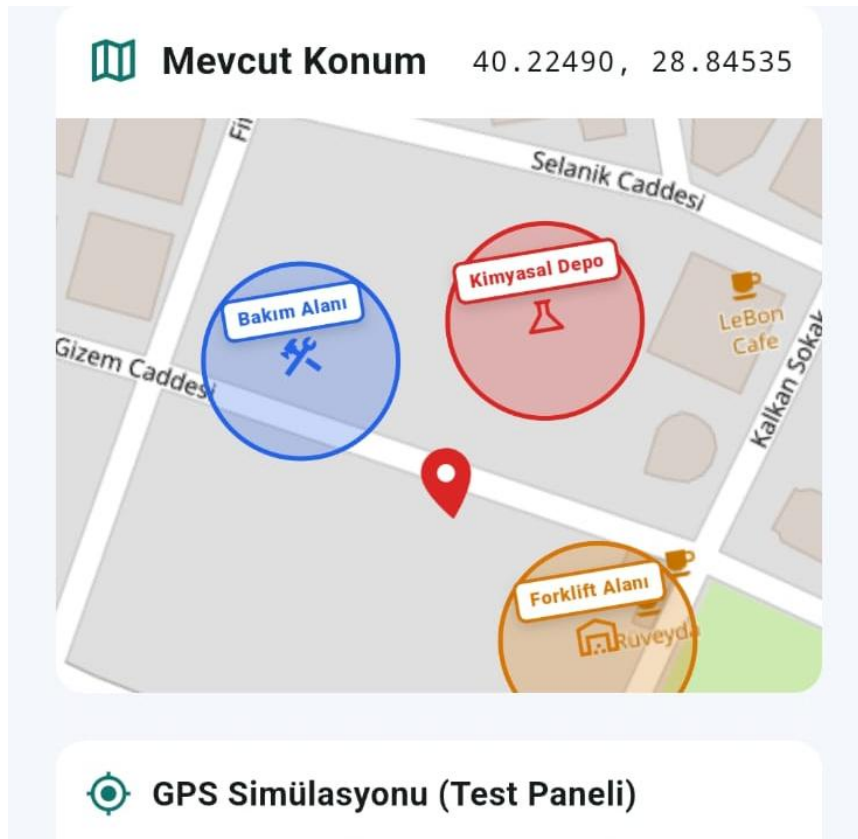
Şekil 14. QR Tehlikeli Bölge Girişi Ekranı

Şekil 14. QR Kod ile Tehlikeli Bölge Girişi Ekranı

Şekil 14'te worker kullanıcısının QR kod okutarak tehlikeli bölge girişini sisteme iletlediği ekran gösterilmelidir. Backend QR kod bilgisini DangerZone kaydıyla eşleştirir; bölge risk seviyesi yüksekse DANGER_ZONE_ENTRY alarmı oluşturulur ve bölge giriş kaydı ZoneEntry koleksiyonuna kaydedilir.

7.7. GPS Konum Desteği

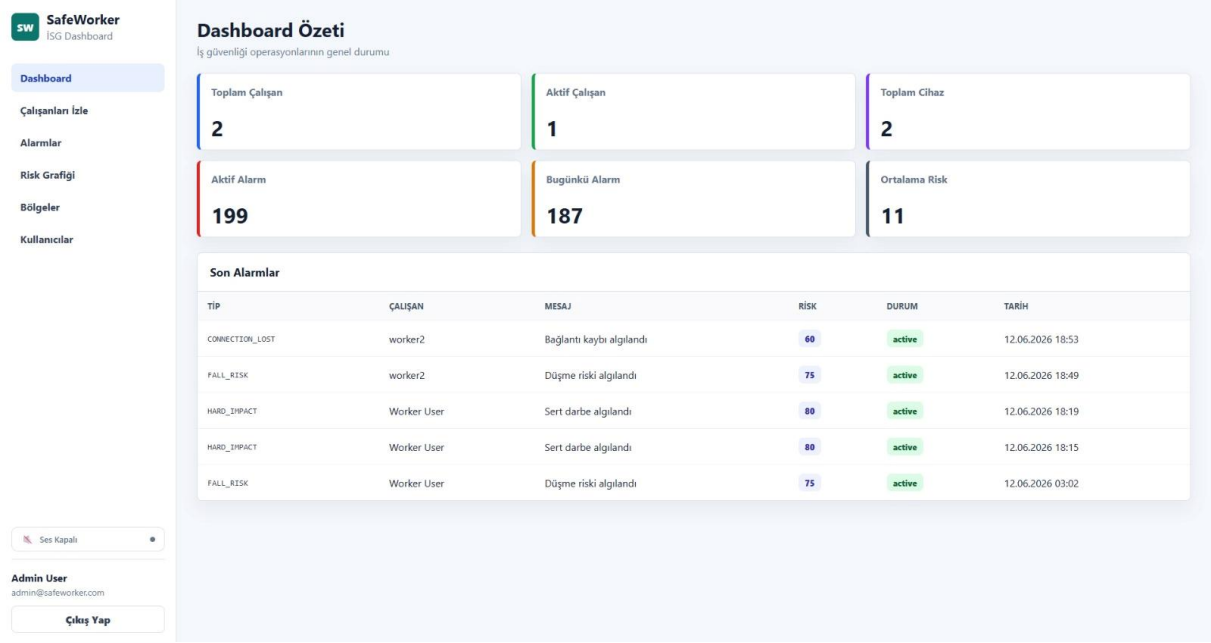
Mobil uygulama GPS konum bilgisini sensör kaydıyla birlikte backend'e gönderebilir. Bu özellik, çalışanın riskli durum yaşadığı konumun daha iyi yorumlanmasına katkı sağlar. Mevcut MVP'de GPS verisi destekleyici veri olarak tutulmaktadır; ileride harita üzerinde canlı konum gösterimi ile geliştirilebilir.



8. Dashboard Tasarımı

8.1. Dashboard Genel Görünüm

Dashboard, Admin kullanıcısının sistemi izlediği web arayüzüdür. React ile geliştirilmiştir ve backend API'leri ile Socket.IO eventlerini birlikte kullanır. Admin dashboard üzerinden sistem özetini, çalışan durumlarını, risk grafiklerini, alarm listesini, tehlikeli bölgeleri ve CSV raporlarını takip edebilir.



Şekil 15. Admin Dashboard Genel Görünüm Ekranı

Şekil 15. Admin Dashboard Genel Görünüm Ekranı

Şekil 15’da admin kullanıcısının SafeWorker sistemindeki genel durumu takip ettiği dashboard ekranı yer almalıdır. Bu ekran, sistem özetinin, aktif çalışanların, alarm durumunun ve risk göstergelerinin tek bir arayüz üzerinden izlenebildiğini göstermek için kullanılacaktır.

8.2. Son Durum

Dashboard üzerinde aktif çalışan, cihaz, aktif alarm, günlük alarm ve ortalama risk puanı gibi özet kartları bulunur. Bu kartlar admin kullanıcısının sistemin genel durumunu hızlıca değerlendirmesini sağlar.

Son Alarmlar					
TİP	ÇALIŞAN	MESAJ	RİSK	DURUM	TARİH
CONNECTION_LOST	worker2	Bağlantı kaybı algılandı	60	active	12.06.2026 18:53
FALL_RISK	worker2	Düşme riski algılandı	75	active	12.06.2026 18:49
HARD_IMPACT	Worker User	Sert darbe algılandı	80	active	12.06.2026 18:19
HARD_IMPACT	Worker User	Sert darbe algılandı	80	active	12.06.2026 18:15
FALL_RISK	Worker User	Düşme riski algılandı	75	active	12.06.2026 03:02

Şekil 16. Son Durum Ekranı

Şekil 16. Son Durum Ekranı

Şekil 16’de dashboard üzerindeki toplam çalışan, aktif çalışan, toplam cihaz, aktif alarm, günlük alarm ve ortalama risk puanı kartları gösterilmelidir. Bu görsel, admin kullanıcısının sistemin genel durumunu hızlıca değerlendirebildiğini ve risk takibini özet metriklerle yapabildiğini göstermektedir.

8.3. Çalışan Takibi

Çalışan takibi ekranında worker’ın aktiflik durumu, son görülme zamanı, pil seviyesi, bağlantı durumu ve risk seviyesi gibi bilgiler gösterilir. worker:status ve sensor:new olayları sayesinde bu bilgiler sayfa yenilenmeden güncellenebilir.

ÇALIŞAN	CİHAZ	SON VERİ	PİL	BAĞLANTI	SON RISK	RISK SEVİYESİ	SON ALARM
Worker User	Worker Phone	12.06.2026 18:37	%45	online	0	normal	Sert darbe algılandı
worker2	worker2 Otomatik Cihaz	12.06.2026 18:53	%57	offline	30	normal	Bağlantı kaybı algılandı

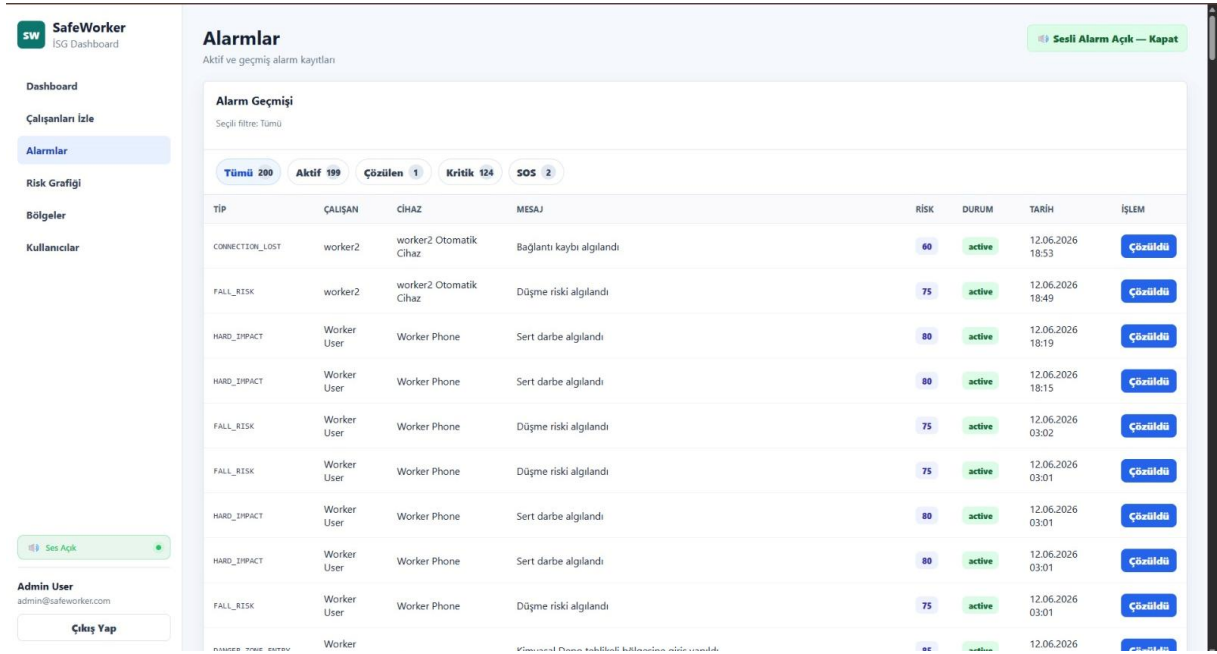
Şekil 17. Çalışan Takibi Ekranı

Şekil 17. Çalışan Takibi Ekranı

Şekil 17’de worker kullanıcılarının aktiflik durumu, son görülme zamanı, pil seviyesi, bağlantı durumu ve risk seviyesi gibi bilgilerin dashboard üzerinden takip edildiği ekran gösterilmelidir. Socket.IO desteği sayesinde worker durumu ve sensör verileri sayfa yenilenmeden güncellenebilir.

8.4. Alarm Listesi ve Filtreleme

Alarm listesi bölümünde oluşan alarmlar tür, risk skoru, durum ve zaman bilgisiyle birlikte görüntülenir. Admin alarmları tümü, aktif, çözölen, kritik ve SOS gibi filtrelerle inceleyebilir. Alarm çözöldüğünde status alanı güncellenir ve alarm geçmişinde çözöldü olarak izlenebilir.



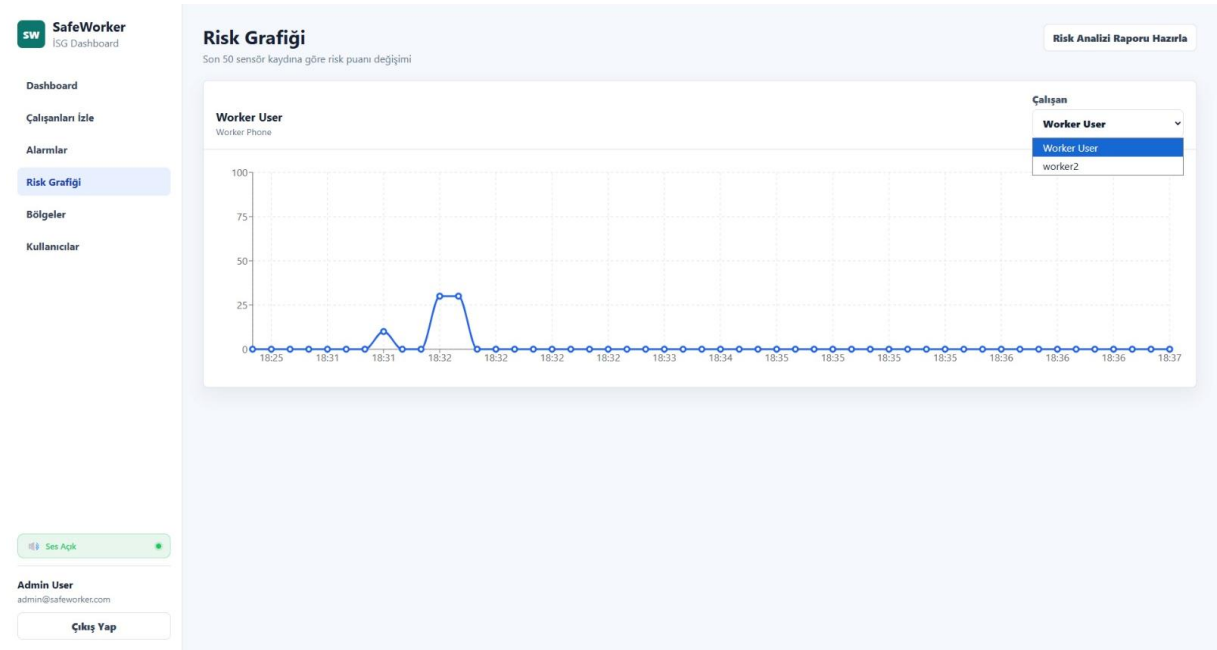
Şekil 18. Alarm Listesi ve Filtreleme Ekranı

Şekil 18. Alarm Listesi ve Filtreleme Ekranı

Şekil 18’da sistemde oluşan alarm kayıtlarının admin tarafından görüntülediği ve filtrelendiği ekran gösterilmelidir. Admin bu ekran üzerinden aktif, çözülen, kritik veya SOS alarm kayıtlarını takip edebilir; gerekli durumda alarmı çözüldü olarak işaretleyebilir.

8.5. Risk Grafiği ve Tablo Gösterimleri

Risk grafiği, çalışanlardan gelen risk puanlarının zaman içindeki değişimini göstermeye yardımcı olur. Tablo gösterimleri ise alarm ve worker verilerinin daha düzenli incelenmesini sağlar. Bu yapı admin kullanıcısının hem genel durumu hem de detay kayıtları takip etmesini kolaylaştırır.



Şekil 19. Risk Grafiği ve Zaman Serisi Gösterimi

Şekil 19. Risk Grafiği ve Zaman Serisi Gösterimi

Şekil 19’de worker kullanıcılarından gelen sensör verileri sonucunda hesaplanan risk puanlarının grafiksel gösterimi yer almalıdır. Grafik, risk seviyesinin zaman içindeki değişimini izlemeyi kolaylaştırır ve f yde beklenen grafik/tablo ile zaman serisi g sterimi şartını destekler.

8.6. Sesli Kritik Alarm Sistemi

Kritik veya SOS alarmı geldiğinde adminin alarmı daha hızlı fark edebilmesi i in sesli kritik alarm sistemi eklenmiřtir. Tarayıcı izin verdi inde sesli uyarı  alıřır. Bu  zellik, adminin s rekli ekrana bakmadı ı durumlarda kritik olayların fark edilmesine katkı sa lar.

9. Risk Analizi ve Alarm Algoritması

9.1. Eşik Tabanlı Risk Analizi

SafeWorker MVP’de risk analizi için makine öğrenmesi yerine eşik tabanlı bir yaklaşım tercih edilmiştir. Bunun nedeni, MVP aşamasında eşik tabanlı algoritmanın daha anlaşılır, test edilebilir ve açıklanabilir olmasıdır. Mobil cihazdan gelen ivmeölçer, jiroskop, pil, ağ ve konum verileri backend tarafında değerlendirilir. Risk faktörleri belirli puanlarla risk skoruna eklenir ve toplam puana göre risk seviyesi belirlenir.

Backend’e gelen sensör verisi doğrulandıktan sonra risk puanı başlatılır. Sert darbe, düşme riski, olay sonrası hareketsizlik, düşük pil ve bağlantı kaybı gibi risk faktörleri varsa risk puanı artırılır. Hesaplanan puana göre risk seviyesi normal, warning veya danger olarak belirlenir. Risk yüksek veya kritik olay varsa alarm oluşturulur ve MongoDB’ye kaydedilir. Son aşamada sensör veya alarm bilgisi Socket.IO ile dashboard’a iletilir.

9.2. Risk Puanı Hesaplama

Risk Faktörü	Puan	Açıklama
Sert darbe	+40	Ani ve yüksek ivme algılandığında eklenir.
Düşme riski	+35	Yüksek ivme ve yüksek jiroskop hareketi birlikte algılandığında eklenir.
Olay sonrası hareketsizlik	+10	Sert darbe/düşme şüphesinden sonra uzun süre hareket yoksa eklenir.
Düşük pil	+10	Pil seviyesi kritik eşik altına düştüğünde eklenir.
Bağlantı kaybı	+30	Cihaz çevrimdışı olduğunda veya veri akışı kesildiğinde eklenir.
Normal hareketsizlik	+0	Tek başına alarm sebebi değildir.

9.3. Risk Seviyeleri

Risk Puanı Aralığı	Risk Seviyesi	Açıklama
0-30	normal	Beklenen güvenli çalışma durumu.
31-60	warning	Dikkat gerektiren orta risk durumu.
61-100	danger	Alarm oluşturabilecek yüksek risk durumu.

9.4. Alarm Türleri

Alarm Türü	Ne Zaman Oluşur?
HARD IMPACT	Ani ve yüksek ivme algılandığında oluşur.
FALL RISK	Yüksek ivme ve jiroskop hareketi birlikte görüldüğünde oluşur.
INACTIVITY	Darbe/düşme sonrası hareketsizlik olduğunda oluşur.
LOW BATTERY	Pil seviyesi kritik eşik altına düştüğünde oluşur.
CONNECTION LOST	Cihaz çevrimdışı olduğunda veya veri akışı kesildiğinde oluşur.
EMERGENCY BUTTON	Worker SOS butonuna bastığında oluşur.
DANGER_ZONE_ENTRY	Yüksek riskli QR bölgeye giriş yapıldığında oluşur.

9.5. Hareketsizlik Mantığının Gerekçesi

Hareketsizlik tek başına alarm sebebi olarak değerlendirilmemiştir. Çünkü çalışan mola veriyor, oturarak çalışıyor veya sabit bir görev yapıyor olabilir. Bu nedenle hareketsizlik, sert darbe veya düşme şüphesi gibi bir olaydan sonra gerçekleşirse anlamlı risk faktörü olarak kabul edilir. Bu yaklaşım yanlış alarm sayısını azaltır ve sistemin gerçek iş güvenliği senaryosuna daha uygun davranmasını sağlar.

10.Gerçek Zamanlı İzleme

10.1. Socket.IO Kullanımı

Backend tarafında Socket.IO kullanılarak dashboard'a gerçek zamanlı veri aktarımı sağlanır. Yeni sensör verisi geldiğinde sensor:new, yeni alarm oluştuğunda alarm:new ve worker'ın vardiya durumu değiştiğinde worker:status olayı yayınlanır. Dashboard bu olayları dinleyerek tablo ve grafik bileşenlerini günceller.

10.2. Dashboard Canlı Güncelleme

Dashboard canlı güncelleme özelliği sayesinde admin yeni verileri sayfa yenilemeden görebilir. Özellikle alarm:new olayı kritik öneme sahiptir çünkü riskli durum oluştuğunda adminin hızlı müdahale etmesini sağlar. worker:status olayı ise çalışanın aktif/pasif durumunun vardiya başlangıcı ve bitişiyle birlikte dashboard'a yansımısını sağlar.

11.Gerçekleştirilen Modüller ve Bonus Özellikler

Gerçekleştirilen modüller özeti aşağıdaki tabloda verilmiştir. Bu tablo, föyde belirtilen zorunlu modüllerin SafeWorker projesindeki karşılığını ve tamamlanma durumunu tek yerde göstermektedir.

Modül	SafeWorker Projesindeki Karşılığı	Durum
Mobil veri toplama	Flutter mobil uygulama ile ivmeölçer, jiroskop, pil, ağ ve GPS verilerinin vardiya aktifken otomatik gönderilmesi.	Gerçekleştirildi
Node.js backend	Express.js tabanlı REST API, controller-service-model yapısı, veri doğrulama ve hata yönetimi.	Gerçekleştirildi
Kullanıcı doğrulama ve yetkilendirme	JWT, bcrypt ve admin/worker rol ayrımı.	Gerçekleştirildi
Veritabanı	MongoDB üzerinde User, Device, Shift, SensorData, Alarm, DangerZone ve ZoneEntry koleksiyonları.	Gerçekleştirildi
Gerçek zamanlı izleme paneli	React dashboard ve Socket.IO ile canlı çalışan, alarm ve risk takibi.	Gerçekleştirildi
Analiz / anomali tespiti	Eşik tabanlı risk puanı, risk seviyesi ve alarm üretim algoritması.	Gerçekleştirildi
Alarm / bildirim mekanizması	HARD_IMPACT, FALL_RISK, INACTIVITY, LOW_BATTERY, CONNECTION_LOST, EMERGENCY_BUTTON ve DANGER_ZONE_ENTRY alarmları.	Gerçekleştirildi
Dokümantasyon	README, Swagger/OpenAPI, teknik rapor, test senaryoları ve kurulum açıklamaları.	Gerçekleştirildi

Bonus Özellik	Projeye Katkısı
Swagger / OpenAPI	API endpointleri anlaşılır ve test edilebilir hale getirilmiştir.
Docker / Docker Compose	MongoDB, backend ve dashboard servislerinin birlikte çalıştırılması desteklenmiştir.
Socket.IO canlı veri akışı	Dashboard anlık verilerle güncellenmektedir.
GPS konum desteği	Worker'ın konum bilgisi sensör kaydıyla birlikte tutulmaktadır.
CSV alarm dışı aktarma	Alarm geçmişi raporlanabilir hale getirilmiştir.
Sesli kritik alarm	Admin kritik ve SOS alarmlarını ekrana bakmadan fark edebilir.
Gelişmiş alarm filtreleme	Alarm yönetimi daha kullanılabilir hale getirilmiştir.
Rol bazlı erişim kontrolü	Admin ve worker işlemleri ayrılmıştır.
QR tehlikeli bölge sistemi	Riskli alan girişleri izlenebilir ve alarm üretilebilir hale getirilmiştir.
Vardiya bazlı veri gönderimi	Worker vardiyası başlamadan sensör veri akışı başlamaz, vardiya bitince durur.

12.Kurulum ve Çalıştırma

12.1. Gereksinimler

- Node.js ve npm
- MongoDB veya Docker ile MongoDB servisi
- Git / GitHub
- Flutter SDK ve Android Studio ya da fiziksel Android cihaz
- İsteğe bağlı Docker Desktop
-

12.2. Backend Kurulumu

Backend klasörüne geçildikten sonra bağımlılıklar npm install komutu ile yüklenir. .env dosyasında MongoDB bağlantı adresi, JWT secret ve gerekli port bilgileri tanımlanır. Test verileri için npm run seed komutu çalıştırılabilir. Ardından npm run dev komutu ile backend geliştirme modunda başlatılır. Backend varsayılan olarak 3000 portunda çalışacak şekilde yapılandırılmıştır.

12.3. Dashboard Kurulumu

Dashboard klasörüne geçildikten sonra npm install komutu ile bağımlılıklar yüklenir. Daha sonra npm run dev komutu ile React/Vite dashboard başlatılır. Dashboard genellikle 5173 portunda çalışır. Dashboard'un backend'e erişebilmesi için API URL ayarlarının doğru yapılması gerekir.

12.4. Mobil Uygulama Kurulumu

Mobil klasöründe flutter pub get komutu ile Flutter bağımlılıkları yüklenir. Uygulama fiziksel Android cihazda veya emulator üzerinde flutter run komutu ile çalıştırılır. Fiziksel cihaz kullanıldığında telefon ile bilgisayarın aynı Wi-Fi ağında olması ve mobil uygulamanın backend IP adresine yönlendirilmesi gerekir.

12.5. Docker ile Çalıştırma

Proje Docker Compose desteği içermektedir. Docker kurulu bir ortamda docker compose up --build komutu ile MongoDB, backend ve dashboard servislerinin birlikte çalıştırılması hedeflenmiştir. Docker desteği, projenin farklı ortamlarda daha kolay ayağa kaldırılabilmesi için eklenmiş bir bonus özelliktir.

13. Test Süreci ve Doğrulama

13.1. Test Ortamı

Bileşen	Test Ortamı / Bilgi
Bilgisayar	Windows / VS Code / GitHub Desktop
Backend	Node.js + Express, port 3000
Veritabanı	MongoDB local veya Docker MongoDB
Dashboard	React + Vite, port 5173
Mobil	Flutter uygulama, fiziksel Android cihaz veya emulator
Ağ	Telefon ve bilgisayar aynı Wi-Fi üzerinde

13.2. Test Kullanıcıları

Rol	E-posta	Şifre	Amaç
Admin	admin@safeworker.com	123456	Dashboard testi.
Worker	worker@safeworker.com	123456	Mobil uygulama testi.

13.3. Test Senaryoları

Test No	Senaryo	Beklenen Sonuç	Gerçek Sonuç
T-01	Admin login	Dashboard açılır.	Başarılı
T-02	Worker login	Mobil ana ekran açılır.	Başarılı
T-03	Vardiya başlatma	Aktif vardiya oluşur, sensör takibi başlar.	Başarılı
T-04	Normal sensör verisi	Risk normal olur, alarm oluşmaz.	Başarılı
T-05	Sert darbe	HARD IMPACT alarmı oluşur.	Başarılı
T-06	Düşme riski	FALL RISK alarmı oluşur.	Başarılı
T-07	Olay sonrası hareketsizlik	INACTIVITY alarmı oluşur.	Başarılı
T-08	SOS / acil durum	EMERGENCY_BUTTON alarmı ve risk skoru 100 oluşur.	Başarılı
T-09	QR tehlikeli bölge girişi	DANGER_ZONE_ENTRY alarmı oluşur.	Başarılı
T-10	Canlı dashboard güncellemesi	Yeni veri/alarm dashboard'a anlık yansır.	Başarılı
T-11	CSV alarm export	CSV dosyası indirilir veya endpoint çıktı verir.	Başarılı
T-12	Swagger API dokümantasyonu	API dokümantasyonu açılır.	Başarılı

14.Ekip Görev Dağılımı

Proje ekip çalışması olarak geliştirilmiştir. Aşağıdaki görev dağılımı, raporda projenin farklı bileşenlerinin ekip üyeleri arasında nasıl paylaşıldığını göstermek amacıyla verilmiştir. Gerekirse ekip üyelerinin gerçek katkılarına göre güncellenebilir.

Ekip Üyesi	Görev Alanı	Yapılan İşler
Elif Nur Beycan	Backend / genel proje yapısı	Node.js backend, MongoDB bağlantıları, API yapısı ve GitHub proje yönetimi.
Nermin Baycan	Dashboard / görsel takip	React dashboard, canlı çalışan ekranları, alarm listesi ve grafiklerin takibi.
Fatma Nur Yazıcı	Test / dokümantasyon desteği	Test senaryoları, ekran görüntüleri, rapor ve teslim dosyaları desteği.
Kübra Kaya	Mobil arayüz, entegrasyon ve rapor	Mobil worker akışı, vardiya-sensör mantığı, diyagramlar, rapor ve test süreci.

15.Sonuç ve Değerlendirme

15.1. Projenin Genel Başarısı

SafeWorker MVP, föyde beklenen temel modüllerin büyük kısmını karşılayan çalışır bir iş güvenliği mobil risk algılama sistemidir. Sistem mobil cihazdan veri toplamakta, Node.js backend üzerinde bu verileri analiz etmekte, MongoDB’de kayıt altına almakta, riskli durumlarda alarm üretmekte ve React dashboard üzerinde gerçek zamanlı olarak göstermektedir. Admin ve worker rollerinin ayrılması, vardiya bazlı veri gönderimi, QR tehlikeli bölge girişi ve SOS bildirimi projeyi daha gerçekçi bir iş güvenliği senaryosuna yaklaştırmıştır.

15.2. Kazanımlar

Bu proje kapsamında Node.js ve Express.js ile REST API geliştirme, MongoDB ve Mongoose ile veri modelleme, JWT ile kimlik doğrulama, React dashboard tasarımı, Flutter mobil uygulama entegrasyonu, Socket.IO ile gerçek zamanlı veri aktarımı, Swagger API dokümantasyonu ve Docker destekli kurulum konularında uygulamalı deneyim kazanılmıştır. Ayrıca sistem mimarisi, veri akışı, ER diyagramı, sequence diyagramı ve risk analizi akışı gibi yazılım mühendisliği dokümantasyonları hazırlanmıştır.

15.3. Gelecek Geliştirmeler

- Offline veri tamponlama geliştirilerek bağlantı geldiğinde verilerin toplu gönderilmesi sağlanabilir.
- Makine öğrenmesi veya yapay zeka modeli ile daha gelişmiş risk sınıflandırması yapılabilir.
- Kamera ile olay fotoğrafı ekleme özelliği geliştirilebilir.
- Yetki yönetimi daha detaylı hale getirilebilir.
- Gerçek saha testleri ve performans ölçümleri yapılabilir.
- Alarm önceliklendirme ve bildirim geçmişi daha gelişmiş hale getirilebilir.
- Vardiya başlatma/durdurma tuşlamasını QR kod ile gerçekleştirme.

Kaynakça

- [1] Dijital Kaynak: Google. (t.y.). Flutter documentation. Flutter. Erişim Tarihi: 12.06.2026, Erişim Adresi: <https://docs.flutter.dev/>
- [2] Dijital Kaynak: Node.js. (t.y.). Node.js documentation. Node.js. Erişim Tarihi: 12.06.2026, Erişim Adresi: <https://nodejs.org/docs/latest/api/>
- [3] Dijital Kaynak: Express.js. (t.y.). Express - Node.js web application framework. Express.js. Erişim Tarihi: 12.06.2026, Erişim Adresi: <https://expressjs.com/>
- [4] Dijital Kaynak: Meta Open Source. (t.y.). React documentation. React. Erişim Tarihi: 12.06.2026, Erişim Adresi: <https://react.dev/>
- [5] Dijital Kaynak: MongoDB, Inc. (t.y.). MongoDB documentation. MongoDB. Erişim Tarihi: 12.06.2026, Erişim Adresi: <https://www.mongodb.com/docs/>
- [6] Dijital Kaynak: Mongoose. (t.y.). Mongoose documentation. Mongoose. Erişim Tarihi: 12.06.2026, Erişim Adresi: <https://mongoosejs.com/docs/>
- [7] Dijital Kaynak: Socket.IO. (t.y.). Socket.IO documentation. Socket.IO. Erişim Tarihi: 12.06.2026, Erişim Adresi: <https://socket.io/docs/v4/>
- [8] Dijital Kaynak: SmartBear Software. (t.y.). Swagger documentation. Swagger. Erişim Tarihi: 12.06.2026, Erişim Adresi: <https://swagger.io/docs/>
- [9] Dijital Kaynak: Docker, Inc. (t.y.). Docker documentation. Docker Docs. Erişim Tarihi: 12.06.2026, Erişim Adresi: <https://docs.docker.com/>
- [10] Dijital Kaynak: Beycan, E. N., Baycan, N., Yazıcı, F. N., & Kaya, K. (2026). SafeWorker: İş sağlığı ve güvenliği için mobil risk algılama platformu [GitHub repository]. GitHub. Erişim Adresi: <https://github.com/elifnurbeycan/SafeWorker>
- [11] Ders Dokümanı: Bursa Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü. (2026). Node.js ile Web Programlama dönem projesi föyü: Mobil Güvenlik ve Davranış Analizi Platformu. Bursa Teknik Üniversitesi.
- [12] Mevzuat Kaynağı: T.C. Resmî Gazete. (2012, 30 Haziran). 6331 Sayılı İş Sağlığı ve Güvenliği Kanunu. Resmî Gazete, Sayı: 28339. Erişim Adresi: <https://www.mevzuat.gov.tr/>
- [13] Basılı Kaynak: Podgórski, D., Majchrzycka, K., Dąbrowska, A., Gralewicz, G., & Okrasa, M. (2017). Towards a conceptual framework of OSH risk management in smart working environments based on smart PPE, ambient intelligence and the Internet of Things technologies. International Journal of Occupational Safety and Ergonomics, 23(1), 1-20. <https://doi.org/10.1080/10803548.2016.1214431>
- [14] Basılı Kaynak: Olak, A. J., Hejduk, I., Karwowski, W., Tomczyk, P., Fazlagić, J., Gac, P., Hejduk, H., Sobolewska, S., Çakıt, E., & Alrehaili, O. A. (2021). The relationships between the use of smart mobile technology, safety knowledge and propensity to follow safe practices at work. International Journal of Occupational Safety and Ergonomics, 27(3), 911-920. <https://doi.org/10.1080/10803548.2019.1658398>
- [15] Basılı Kaynak: Lee, W., Lin, K.-Y., Seto, E., & Migliaccio, G. C. (2017). Wearable sensors for monitoring on-duty and off-duty worker physiological status and activities in construction. Automation in Construction, 83, 341-353. <https://doi.org/10.1016/j.autcon.2017.06.012>
- [16] Basılı Kaynak: Zimbelman, E. G., & Keefe, R. F. (2017). Hazards in motion: Development of mobile geofences for use in logging safety. Sensors, 17(4), 822. <https://doi.org/10.3390/s17040822>